POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Project Report
Computer Science and Engineering
Ingegneria Informatica

Polygeist-GVSoC

Author:
Kushagra Varshney
Rajatkant Nayak

Student ID:
11142280
11144180

Advisor:
Prof. Agosta Giovanni

Academic Year:
2025-26

Abstract

We present **Polygeist-GVSoC**, an end-to-end toolchain that compiles OpenMP-annotated C through MLIR to bare-metal RISC-V ELF binaries and executes them on the GVSoC PULP virtual-platform simulator. The core challenge is cross-version integration of four open-source projects— Polygeist (LLVM 18), PULP LLVM (LLVM 15), GVSoC, and PolyBench-ACC—whose incompatible toolchain generations required careful interface engineering at every pipeline stage.

The seven-stage pipeline proceeds as: C → MLIR (`cgeist`) → LLVM dialect (`mlir-opt`) → LLVM IR (`mlir-translate`) → RISC-V assembly (`llc`) → bare-metal binary (`ELF`) → GVSoC simulation.

We implement two bare-metal OpenMP runtimes in this project: a serial fallback for the single-core `rv32` target, and a fully parallel runtime for the `pulp-open` 8-core PULP cluster. The parallel runtime uses a Fabric-Controller dispatcher model with per-core done flags in L2 memory; all synchronization is built on TCDM test-and-set hardware primitives, requiring neither the RI5CY A-extension nor an operating system.

Validation uses four PolyBench-ACC kernels (GEMM, ATAX, 2MM, 3MM) and a 23-test suite covering tool availability, hardware boot, TCDM locks, and OpenMP barriers—all passing on both targets. A Docker build environment ensures full reproducibility.

Keywords: ***MLIR, RISC-V, PULP, GVSoC, OpenMP, bare-metal, Polygeist, LLVM, TCDM.***

Contents

List of Figures

List of Tables

1 | Introduction

The PULP (Parallel Ultra-Low-Power Processing) platform [4] provides a family of open-source RISC-V-based processors targeting energy-efficient computing for IoT and edge AI workloads. GVSoC [1] is a highly configurable, cycle-accurate virtual platform simulator for these processors, enabling functional verification and performance analysis without physical hardware.

Compiling high-level C code to run on bare-metal PULP processors involves multiple non-trivial challenges: generating efficient RISC-V code through multi-level intermediate representations, providing minimal runtime support without an operating system, and handling hardware-specific synchronization mechanisms for multi-core execution.

This project assembles a complete toolchain from independently developed components:

- **Polygeist** [3]: A C/C++ front-end that compiles to MLIR, enabling polyhedral analysis and optimization via MLIR's affine and OpenMP dialects.
- **PULP LLVM** [5]: A fork of LLVM with RISC-V extensions for PULP processors, including Xpulpv2 (hardware loops, post-increment loads, MAC instructions) and Snitch extensions (SSR, FREP, DMA).
- **GVSoC** [1]: A full-platform simulator supporting multiple PULP targets including single-core `rv32` and multi-core `pulp-open` with RI5CY cluster cores.
- **PolyBench-ACC** [2]: A collection of parallel benchmark kernels with OpenMP annotations, providing validated computational patterns for toolchain verification.

1.1. Project Objectives

The primary objectives of this project are:

1. Integrate the complete C → MLIR → LLVM IR → RISC-V → GVSoC toolchain from source.
2. Implement bare-metal runtime support for both single-core and multi-core GVSoC targets.
3. Implement OpenMP runtime libraries that correctly handle the function calls generated by MLIR's OpenMP lowering.
4. Validate the toolchain using real benchmark kernels from PolyBench-ACC, executing them on GVSoC and verifying correctness via checksum return values.

Open-Source Contribution: The complete toolchain, runtime implementations, and automated test suite developed for this project are publicly available at `https://github.com/Kush3012/polygeist-gvsoc`.

1.2. Report Organization

In this report chapter 2 presents the toolchain components (Polygeist, PULP LLVM, GVSoC, PolyBench-ACC) and describes the seven-step compilation pipeline. Chapter 3 details the technical challenges and integration issues encountered during development, including compiler version compatibility, MLIR pass ordering sensitivities, and a critical GVSoC simulator bug. Chapter 4 documents the experimental validation, presenting benchmark results on both single-

core and multi-core targets, cycle-level VCD waveform analysis, and an investigation into GAP8 and GAP9 target availability. Chapter 5 outlines the toolchain automation artifacts and presents the final conclusions.

1.3. Architecture Overview

The toolchain follows a seven-step pipeline from C source to simulated execution, as illustrated in Figure 1.1.

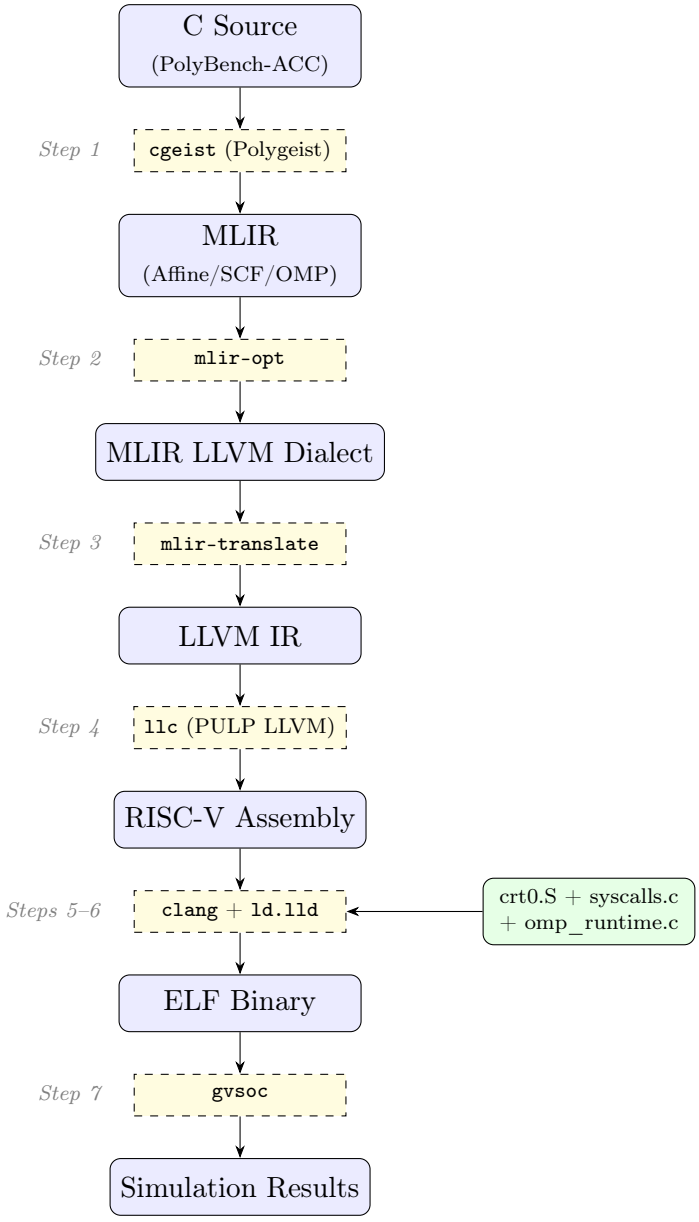

Figure 1.1: Seven-step compilation pipeline from C source to GVSoC execution.

Each stage transforms the code through progressively lower levels of abstraction:

1. C source → MLIR with affine/SCF/OpenMP dialects
2. MLIR high-level dialects → MLIR LLVM dialect
3. MLIR LLVM dialect → LLVM IR (`.ll` textual form)
4. LLVM IR → RISC-V assembly (`.s`)
5. Assembly → relocatable object (`.o`)
6. Object + runtime → linked ELF binary
7. ELF → GVSoC simulated execution

2 | Toolchain Design and Compilation Pipeline

2.1. Polygeist (C to MLIR)

Polygeist[1] is an MLIR-based C/C++ compiler front-end developed at MIT. It leverages Clang's parsing infrastructure to generate an Abstract Syntax Tree (AST) and then translates the AST into MLIR operations. Unlike traditional Clang-to-LLVM compilation, Polygeist preserves high-level loop structure in the `affine` and `scf` MLIR dialects, enabling polyhedral analysis, loop transformations, and OpenMP parallelization at the MLIR level.

Version: Polygeist main branch (commit `77c04bb`) with LLVM submodule at commit `26eb428` (LLVM 18-based).

Key tool: `cgeist` — the C-to-MLIR front-end compiler.

MLIR dialects produced:

- `affine`: Polyhedral loop representation with `affine.for` and `affine.if`
- `scf`: Structured control flow (`scf.for`, `scf.while`, `scf.if`)
- `omp`: OpenMP dialect (`omp.parallel`, `omp.wsloop`) when `-fopenmp` is passed
- `memref`: Multi-dimensional memory references
- `arith`: Arithmetic operations (integer and floating-point)
- `func`: Function definitions and calls

Important: Polygeist's LLVM 18 submodule requires GCC 14. GCC 15 fails due to missing `#include <cstdint>` in certain LLVM headers. GCC 12 fails with C++17 standard library incompatibilities.

2.2. PULP LLVM (LLVM IR to RISC-V)

The PULP platform maintains a fork of LLVM[2] with custom extensions for their RISC-V processors:

- **Xpulpv2**: Hardware loops (`lp.setup`, `lp.count`), post-increment memory accesses (`lw_postinc`), multiply-accumulate (`p.mac`), and SIMD operations for RI5CY cores.
- **Xssr/Xfrep/Xdma**: Stream Semantic Registers, floating-point repetition, and DMA extensions for Snitch cores.

Version: Tag `15.0.0-snitch-0.5.0` (LLVM 15-based).

Why LLVM 15: Polygeist (LLVM 18) generates LLVM IR with *opaque pointers* (`ptr` instead of `i32*`). LLVM 12 does not support opaque pointers. LLVM 15 supports both typed and opaque pointer syntax, making it compatible with Polygeist's output while providing PULP-specific backend features.

[1] `https://github.com/llvm/Polygeist`
[2] `https://github.com/pulp-platform/llvm-project`

2.3. GVSoC (PULP Platform Simulator)

GVSoC[3] is a cycle-accurate virtual platform simulator for PULP processors. It models the complete SoC including processor cores, memory hierarchy, interconnects, and peripherals.

We use two GVSoC targets:

- `rv32`: A minimal single-core RV32IMFDC simulator with 16 MB of memory at `0x80000000`. It uses HTIF (Host-Target Interface) for program exit signaling via a bus watchpoint at `0x80001000`.
- `pulp-open`: The PULP-open SoC with a Fabric Controller (FC) and an 8-core RI5CY cluster with shared L1 Tightly-Coupled Data Memory (TCDM). Uses the RISC-V semihosting protocol for exit signaling.

2.4. PolyBench-ACC (Benchmarks)

PolyBench-ACC[4] provides parallel versions of the PolyBench benchmark suite across multiple programming models (OpenMP, OpenACC, CUDA, OpenCL). We use the **OpenMP** variant for its portability and direct compatibility with MLIR's OpenMP dialect lowering.

The PolyBench suite covers diverse computational patterns:

- **Linear algebra kernels**: gemm, 2mm, 3mm, atax, bicg, gemver, gesummv, mvt, symm, syr2k, syrk, trisolv, trmm
- **Linear algebra solvers**: durbin, gramschmidt, lu, ludcmp
- **Stencils**: adi, fdtd-2d, jacobi-1d/2d, seidel-2d
- **Datamining**: correlation, covariance
- **Medley**: floyd-warshall, reg_detect

From this suite, we selected and adapted four kernels—GEMM, ATAX, 2MM, and 3MM representing a range of data-parallel patterns from simple matrix multiplication to chained multi-matrix computations.

[3] `https://github.com/gvsoc/gvsoc`
[4] `https://github.com/cavazos-lab/PolyBench-ACC`

2.5. Compilation Pipeline

This chapter describes each of the seven compilation stages in detail, including the exact commands, flags, and transformations involved.

2.5.1. Step 1: C to MLIR (`cgeist`)

The `cgeist` tool parses C source using Clang's front-end and generates MLIR output. Two modes are supported:

Without OpenMP (serial mode):

```
cgeist -S -function=* -raise-scf-to-affine \
  input.c -o output.mlir
```

With OpenMP (parallel mode):

```
cgeist -S -function=* -raise-scf-to-affine -fopenmp \
  input.c -o output.mlir
```

Key flags:

- `-S`: Emit textual MLIR (not binary).
- `-function=*`: Compile all functions (not just `main`).
- `-raise-scf-to-affine`: Promote SCF loops to affine loops where possible, enabling polyhedral optimization.
- `-fopenmp`: Preserve OpenMP pragmas as `omp.parallel` and `omp.wsloop` MLIR operations.

```
$ echo "[1/7] C -> MLIR (cgeist)..."
[1/7] C -> MLIR (cgeist)...
$ grep -n -A 15 "omp.parallel" gemm_kernel.mlir
51:    omp.parallel   {
52-      %2 = arith.index_cast %arg0 : i32 to index
53-      omp.wsloop   for  (%arg8) : index = (%c0) to (%2) step (%c1) {
54-        affine.for %arg9 = 0 to %0 {
55-          %3 = memref.load %arg5[%arg8, %arg9] : memref<?x32xf32>
56-          %4 = arith.mulf %3, %arg4 : f32
57-          memref.store %4, %arg5[%arg8, %arg9] : memref<?x32xf32>
58-        }
59-        affine.for %arg9 = 0 to %1 {
60-          affine.for %arg10 = 0 to %0 {
61-            %3 = memref.load %arg6[%arg8, %arg9] : memref<?x32xf32>
62-            %4 = arith.mulf %arg3, %3 : f32
63-            %5 = affine.load %arg7[%arg9, %arg10] : memref<?x32xf32>
64-            %6 = arith.mulf %4, %5 : f32
65-            %7 = memref.load %arg5[%arg8, %arg10] : memref<?x32xf32>
66-            %8 = arith.addf %7, %6 : f32
```

Figure 2.1: Frontend Parsing (`cgeist`): Translating the GEMM kernel into MLIR affine loops and OpenMP dialects.

2.5.2. Step 2: MLIR Optimization and Lowering (`mlir-opt`)

```
$ echo "[2/7] MLIR lowering (mlir-opt)..."
[2/7] MLIR lowering (mlir-opt)...
$ echo "Applying: --lower-affine --convert-scf-to-cf --convert-openmp-to-llvm ..."
Applying: --lower-affine --convert-scf-to-cf --convert-openmp-to-llvm ...
$ grep -n "llvm.func" gemm_kernel_lowered.mlir
2:  llvm.func @init_array(%arg0: i32, %arg1: i32, %arg2: i32,
      %arg3: !llvm.ptr, %arg4: !llvm.ptr, %arg5: !llvm.ptr,
      %arg6: !llvm.ptr, %arg7: !llvm.ptr) {
144:  llvm.func @kernel_gemm(%arg0: i32, %arg1: i32, %arg2: i32,
      %arg3: f32, %arg4: f32, %arg5: !llvm.ptr,
      %arg6: !llvm.ptr, %arg7: !llvm.ptr) {
246:  llvm.func @main() -> i32 {
```

Figure 2.2: MLIR Lowering (`mlir-opt`): Transforming high-level dialects into the standard LLVM dialect structure.

Critical pass ordering: The `-convert-scf-to-cf` pass *must* come *before* `-convert-openmp-to-llvm`. Reversing this order causes `unrealized_conversion_cast` legalization failures because the OpenMP lowering pass generates `index`-typed loop bounds that the subsequent SCF-to-CF conversion cannot reconcile. The error manifests as:

```
error: failed to legalize operation
  'builtin.unrealized_conversion_cast'
```

2.5.3. Step 3: MLIR to LLVM IR (`mlir-translate`)

```
mlir-translate --mlir-to-llvmir input_llvm.mlir -o output.ll
```

This step translates the MLIR LLVM dialect (which uses MLIR's type system) into textual LLVM IR (`.ll` format) that can be consumed by `llc`. The translation is largely a 1:1 mapping because the MLIR LLVM dialect was specifically designed to mirror the LLVM IR instruction set.

After translation, the target triple and data layout must be patched because `cgeist` defaults to the host architecture (`x86_64-unknown-linux-gnu`):

```
sed -i 's|target triple = ".*"|target triple = \
  "riscv32-unknown-elf"|' output.ll
sed -i 's|target datalayout = ".*"|target datalayout = \
  "e-m:e-p:32:32-i64:64-n32-S128"|' output.ll
```

The data layout string specifies:

- `e`: Little-endian byte order
- `m:e`: ELF name mangling
- `p:32:32`: 32-bit pointers with 32-bit alignment
- `i64:64`: 64-bit integers aligned to 64 bits
- `n32`: 32-bit native integer width
- `S128`: 128-bit stack natural alignment

Without this fix, `llc` interprets the LLVM IR as targeting x86 and generates x86 assembly instead of RISC-V.

```
$ echo "[3/7] MLIR -> LLVM IR (mlir-translate)..."
[3/7] MLIR -> LLVM IR (mlir-translate)...
$ grep -n -C 2 "target triple" gemm_kernel.ll
2-source_filename = "LLVMDialectModule"
3-target datalayout = "e-m:e-p:32:32-i64:64-n32-S128"
4:target triple = "riscv32-unknown-elf"
5-
$ grep -n "kmpc_fork_call" gemm_kernel.ll
217:  call void (ptr, i32, ptr, ...) @__kmpc_fork_call(
        ptr @1, i32 1, ptr @kernel_gemm..omp_par, ptr %structArg)
```

Figure 2.3: IR Translation (`mlir-translate`): Generating target-specific LLVM IR with OpenMP fork-call injections.

2.5.4. Step 4: LLVM IR to RISC-V Assembly (`llc`)

```
llc -march=riscv32 -mattr=+m,+f,+c -target-abi=ilp32f \
    -opaque-pointers \
    input.ll -o output.s
```

The resulting ISA is `rv32imfc`: a 32-bit RISC-V core with hardware integer multiplication, single-precision floating-point unit, and compressed instruction encoding.

```
$ echo "[4/7] LLVM IR -> RISC-V assembly (llc)..."
[4/7] LLVM IR -> RISC-V assembly (llc)...
$ grep -n -C 8 "fadd.s" gemm_kernel.s
454-  add   a2, a2, s5
455-  flw   ft1, 0(a2)
456-  add   a2, a5, a1
457-  slli  a2, a2, 2
458-  add   a2, a2, s0
459-  flw   ft2, 0(a2)
460-  fmul.s  ft0, fs0, ft0
461-  fmul.s  ft0, ft0, ft1
462:  fadd.s  ft0, ft2, ft0
463-  fsw   ft0, 0(a2)
464-  addi  a2, a1, 1
```

Figure 2.4: Code Generation (`llc`): Emitting RV32IMFC assembly heavily utilizing hardware floating-point instructions.

2.5.5. Step 5: Assembly to Object File (`clang`)

The PULP LLVM cross-compiler assembles the generated code, and the bare-metal runtime support files are compiled separately:

```
# Kernel object
clang --target=riscv32-unknown-elf -march=rv32imfc \
  -mabi=ilp32f -c output.s -o output.o
```

2.5.6. Step 6: Linking to ELF (`ld.lld`)

```
ld.lld -T link.ld -o output.elf \
  crt0.o output.o syscalls.o omp_runtime.o
```

The linker script defines the memory layout, section placement, and symbol definitions required by the bare-metal runtime.

```
$ echo "[5/7] Assemble -> object file (clang)..."
[5/7] Assemble -> object file (clang)...
$ echo "[6/7] Compile runtime & link -> ELF (ld.lld)..."
[6/7] Compile runtime & link -> ELF (ld.lld)...
$ file gemm_kernel.elf
gemm_kernel.elf: ELF 32-bit LSB executable, UCB RISC-V, RVC,
  single-float ABI, version 1 (SYSV), statically linked, not stripped
```

Figure 2.5: Assembly & Linking: Producing a statically linked, bare-metal RISC-V ELF binary with compressed instructions.

2.5.7. Step 7: Execution on GVSoC

```
# Single-core
gvsoc --target=rv32 --binary=output.elf run

# Multi-core cluster
gvsoc --target=pulp-open --binary=output.elf run

# With instruction tracing (for debugging)
gvsoc --target=pulp-open --binary=output.elf --trace=insn run
```

GVSoC loads the ELF binary into simulated memory according to the program headers, sets the program counter to the entry point from the ELF header, and begins instruction-by-instruction execution. The simulator exit code reflects the program's return value: 0 for success, non-zero for failure.

```
$ ./toolchain.sh gemm_kernel.c
================================================
 End-to-End Pipeline: C -> MLIR -> RISC-V -> GVSoC
 Target: rv32 | Input: gemm_kernel.c
================================================
[1/7] C -> MLIR (cgeist)...
[2/7] MLIR lowering (mlir-opt)...
[3/7] MLIR -> LLVM IR (mlir-translate)...
[4/7] LLVM IR -> RISC-V assembly (llc)...
[5/7] Assemble -> object file (clang)...
[6/7] Compile runtime & link -> ELF (ld.lld)...
[7/7] Executing on GVSoC (rv32)...
----------------------------------------
----------------------------------------
Execution Complete! (exit code: 0)
```

Figure 2.6: Single-Core Simulation: Successful execution of the GEMM kernel on the `rv32` GVSoC target.

```
$ ./toolchain.sh gemm_kernel.c pulp-open
================================================
 End-to-End Pipeline: C -> MLIR -> RISC-V -> GVSoC
 Target: pulp-open | Input: gemm_kernel.c
================================================
[1/7] C -> MLIR (cgeist)...
[2/7] MLIR lowering (mlir-opt)...
[3/7] MLIR -> LLVM IR (mlir-translate)...
[4/7] LLVM IR -> RISC-V assembly (llc)...
[5/7] Assemble -> object file (clang)...
[6/7] Compile runtime & link -> ELF (ld.lld)...
[7/7] Executing on GVSoC (pulp-open)...
----------------------------------------
----------------------------------------
Execution Complete! (exit code: 0)
```

Figure 2.7: Multi-Core Simulation: Parallel execution succeeding across the 8-core `pulp-open` cluster.

2.6. Bare-Metal OpenMP Runtime Implementation

To support the `omp.parallel` and `omp.wsloop` directives generated by MLIR, we implemented two custom bare-metal OpenMP runtimes. For the single-core `rv32` target, we developed a serial fallback runtime that processes OpenMP microtasks sequentially.

For the multi-core `pulp-open` target, we implemented a fully parallel fork-join runtime customized for the PULP architecture. Instead of relying on an operating system, the Fabric Controller (FC) acts as a dedicated dispatcher, waking up the 8 Processing Elements (PEs) and distributing loop iterations via static contiguous-chunk scheduling. To achieve ultra-low latency synchronization without standard RISC-V atomic (A) extensions, the runtime exploits the Tightly-Coupled Data Memory (TCDM). We utilized hardware-level test-and-set primitives within the TCDM to implement spin-locks and sense-reversing barriers, while per-core "done" flags in the L2 memory ensure race-free completion detection.

3 | Technical Challenges and Integration Issues

3.1. Issues and Limitations

1. **Compiler version compatibility:** PULP LLVM (LLVM 15) and Polygeist (LLVM 18) may require different GCC versions depending on the host system. On Ubuntu 22.04, the system GCC (12 or 14) worked for both. If build errors occur, specifying `-DCMAKE_C_COMPILER=gcc-{12,14}` explicitly for each component's CMake configuration may be necessary.
2. **PULP LLVM TableGen assertion:** The PULP fork's RISC-V TableGen definitions may contain duplicate entries that trigger a `"Duplicate entry?"` assertion failure during register info generation on some configurations. If this occurs, building with `-DLLVM_ENABLE_ASSERTIONS=OFF` suppresses it. The generated code is functionally correct.
3. **Target triple/datalayout mismatch:** Polygeist's `cgeist` inherits the host system's target triple and data layout. The generated LLVM IR must be post-processed with `sed` to fix the target triple to `riscv32-unknown-elf` and the data layout to 32-bit. Without this fix, `llc` generates host-architecture assembly.
4. **Linker relaxation unsupported:** PULP LLVM's `ld.lld` does not implement RISC-V linker relaxation. All runtime support code must be compiled with `-mno-relax` to suppress emission of `R_RISCV_ALIGN` / `R_RISCV_RELAX` relocations.
5. **GVSoC single register file (fixed):** The default PULP core configuration in GVSoC uses a unified register file (`ISS_SINGLE_REGFILE=1`) that incorrectly aliases floating-point and integer registers. This caused floating-point load/store instructions to silently corrupt integer loop counters. We fixed this by modifying the platform configuration to disable this flag, enforcing the standard RISC-V separate register file model.
6. **MLIR pass ordering sensitivity:** The OpenMP lowering pipeline requires strict pass ordering (`-convert-scf-to-cf` *before* `-convert-openmp-to-llvm`). Reversing the order causes `unrealized_conversion_cast` legalization failures.
7. **No 64-bit software division:** The bare-metal environment lacks the `__divdi3` compiler runtime function for 64-bit integer division. Our 64-bit loop scheduling variants (`__kmpc_for_static_init_8/8u`) cast to 32-bit internally to avoid this dependency. This is safe for the small loop bounds (≤ 32) used in our benchmarks.

4 | Experimental Validation

4.1. Validation and Results

4.1.1. Single-Core Results (rv32)

To establish a baseline, we first attempted to compile and execute the original, unmodified PolyBench-ACC sources on the GVSoC `rv32` single-core simulator. As shown in Table 4.1, all four benchmarks failed to execute, consistently returning Exit Code 134.

Bench.	Matrix	ELF Size	Cores	OpenMP	Exit Code	Status
GEMM	32×32	9,904 B	1	✓	134	FAIL
ATAX	32×32	9,564 B	1	✓	134	FAIL
2MM	32×32	8,684 B	1	✓	134	FAIL
3MM	32×32	8,764 B	1	✓	134	FAIL

Table 4.1: Baseline `rv32` Execution: kernels failing (Exit Code 134) due to memory constraints.

Analyzing Error Code 134

The root cause of Error 134 on the single-core target stems from the interaction between the `rv32` simulator's strict memory constraints and bare-metal execution. While our automated `toolchain.sh` pipeline successfully sanitizes standard I/O calls (stripping `printf` and `fprintf` to prevent basic syscall exceptions), the original PolyBench kernels rely on massive default dataset sizes. As shown in Figure 4.1, attempting to execute these unmodified kernels results in an immediate runtime abort.

When the unmodified kernels attempt to initialize these massive arrays, the standard library triggers underlying memory-allocation system calls to request heap expansion. Because the bare-metal simulator lacks an operating system to handle dynamic memory allocation requests, the platform intercepts this as an invalid operation, throwing a `std::runtime_error` (`bad syscall #0`) and aborting execution.

This hardware limitation mandated our development of a stripped-down benchmark suite. By scaling down the matrix configurations to 32×32 and replacing dynamic allocations with manageable, statically allocated arrays, our adapted suite executed flawlessly within the simulator's boundaries, as evidenced by the successful Exit Code 0 in Figure 4.2. The complete source code configuration for this successful `rv32` execution is provided in Appendix.

```
$ ./toolchain.sh ./PolyBench-ACC/OpenMP/linear-algebra/kernels/gemm/gemm.c
================================================
 End-to-End Pipeline: C -> MLIR -> RISC-V -> GVSoC
 Target: rv32 | Input: .../gemm/gemm.c
================================================
[1/7] C -> MLIR (cgeist)...
[2/7] MLIR lowering (mlir-opt)...
[3/7] MLIR -> LLVM IR (mlir-translate)...
[4/7] LLVM IR -> RISC-V assembly (llc)...
[5/7] Assemble -> object file (clang)...
[6/7] Compile runtime & link -> ELF (ld.lld)...
[7/7] Executing on GVSoC (rv32)...
----------------------------------------
terminate called after throwing an instance of 'std::runtime_error'
  what():  bad syscall #0
Input error: Platform returned an error (exitcode: 134)

# ATAX, 2MM, 3MM produce identical 7-step output and the same failure:
$ ./toolchain.sh .../atax/atax.c  -> bad syscall #0  (exitcode: 134)
$ ./toolchain.sh .../2mm/2mm.c    -> bad syscall #0  (exitcode: 134)
$ ./toolchain.sh .../3mm/3mm.c    -> bad syscall #0  (exitcode: 134)
```

Figure 4.1: The Bare-Metal Syscall Trap: Original PolyBench sources trigger a memory and syscall exception (Exit Code 134).

```
$ ./toolchain.sh gemm_kernel.c
================================================
 End-to-End Pipeline: C -> MLIR -> RISC-V -> GVSoC
 Target: rv32 | Input: gemm_kernel.c
================================================
[1/7] C -> MLIR (cgeist)...
[2/7] MLIR lowering (mlir-opt)...
[3/7] MLIR -> LLVM IR (mlir-translate)...
[4/7] LLVM IR -> RISC-V assembly (llc)...
[5/7] Assemble -> object file (clang)...
[6/7] Compile runtime & link -> ELF (ld.lld)...
[7/7] Executing on GVSoC (rv32)...
----------------------------------------
Execution Complete! (exit code: 0)
```

Figure 4.2: rv32 Validation Success: Successful execution of the memory-optimized kernels on the single-core `rv32` machine instance (Exit Code 0).

4.1.2. Multi-Core Results (pulp-open)

All four benchmarks were executed on the `pulp-open` multi-core cluster target with the parallel OpenMP runtime, distributing work across 8 processing elements:

Bench.	Matrix	ELF Size	Cores	OpenMP	Exit Code	Status
GEMM	32×32	11,504 B	8	✓	0	PASS
ATAX	32×32	11,164 B	8	✓	0	PASS
2MM	32×32	10,288 B	8	✓	0	PASS
3MM	32×32	10,368 B	8	✓	0	PASS

Table 4.2: Multi-core benchmark results (`pulp-open` target, 8 PEs, parallel OpenMP runtime).

The ELF sizes for `pulp-open` are ∼1.5–2 KB larger than `rv32` due to:

- The parallel OpenMP runtime code (TCDM synchronization, worker entry loop, per-core scheduling, microtask dispatch logic)
- The more complex startup assembly (`crt0_pulp.S` with both FC and PE boot paths)
- The semihosting exit handler (more instructions than the simple HTIF write)

```
$ ./toolchain.sh ./PolyBench-ACC/OpenMP/linear-algebra/kernels/gemm/gemm.c pulp-open
================================================
 End-to-End Pipeline: C -> MLIR -> RISC-V -> GVSoC
 Target: pulp-open | Input: ./PolyBench-ACC/OpenMP/linear-algebra/kernels/gemm/gemm.c
================================================
[1/7] C -> MLIR (cgeist)...
[2/7] MLIR lowering (mlir-opt)...
[3/7] MLIR -> LLVM IR (mlir-translate)...
[4/7] LLVM IR -> RISC-V assembly (llc)...
[5/7] Assemble -> object file (clang)...
[6/7] Compile runtime & link -> ELF (ld.lld)...
[7/7] Executing on GVSoC (pulp-open)...
----------------------------------------
Execution Complete! (exit code: 0)

$ ./toolchain.sh ./PolyBench-ACC/OpenMP/linear-algebra/kernels/atax/atax.c pulp-open
================================================
 End-to-End Pipeline: C -> MLIR -> RISC-V -> GVSoC
 Target: pulp-open | Input: ./PolyBench-ACC/OpenMP/linear-algebra/kernels/atax/atax.c
================================================
[1/7] C -> MLIR (cgeist)...
[2/7] MLIR lowering (mlir-opt)...
[3/7] MLIR -> LLVM IR (mlir-translate)...
[4/7] LLVM IR -> RISC-V assembly (llc)...
[5/7] Assemble -> object file (clang)...
[6/7] Compile runtime & link -> ELF (ld.lld)...
[7/7] Executing on GVSoC (pulp-open)...
----------------------------------------
Execution Complete! (exit code: 0)

$ ./toolchain.sh ./PolyBench-ACC/OpenMP/linear-algebra/kernels/2mm/2mm.c pulp-open
================================================
 End-to-End Pipeline: C -> MLIR -> RISC-V -> GVSoC
 Target: pulp-open | Input: ./PolyBench-ACC/OpenMP/linear-algebra/kernels/2mm/2mm.c
================================================
[1/7] C -> MLIR (cgeist)...
[2/7] MLIR lowering (mlir-opt)...
[3/7] MLIR -> LLVM IR (mlir-translate)...
[4/7] LLVM IR -> RISC-V assembly (llc)...
[5/7] Assemble -> object file (clang)...
[6/7] Compile runtime & link -> ELF (ld.lld)...
[7/7] Executing on GVSoC (pulp-open)...
----------------------------------------
Execution Complete! (exit code: 0)

$ ./toolchain.sh ./PolyBench-ACC/OpenMP/linear-algebra/kernels/3mm/3mm.c pulp-open
================================================
 End-to-End Pipeline: C -> MLIR -> RISC-V -> GVSoC
 Target: pulp-open | Input: ./PolyBench-ACC/OpenMP/linear-algebra/kernels/3mm/3mm.c
================================================
[1/7] C -> MLIR (cgeist)...
[2/7] MLIR lowering (mlir-opt)...
[3/7] MLIR -> LLVM IR (mlir-translate)...
[4/7] LLVM IR -> RISC-V assembly (llc)...
[5/7] Assemble -> object file (clang)...
[6/7] Compile runtime & link -> ELF (ld.lld)...
[7/7] Executing on GVSoC (pulp-open)...
----------------------------------------
Execution Complete! (exit code: 0)
```

Figure 4.3: Parallel Validation: All four adapted benchmarks executing successfully across the 8-core cluster.

4.1.3. Correctness Verification

Correctness is verified at two levels:

1. **Checksum return:** Each benchmark computes a floating-point checksum over the result array by summing all elements. A positive checksum indicates successful computation and triggers exit code 0; a zero or negative checksum indicates failure and triggers exit code 1. The checksum also prevents the compiler from eliminating the kernel computation as dead code.
2. **Cross-target consistency:** The same benchmark with the same deterministic input produces the same validation result (exit code 0) on both `rv32` (serial) and `pulp-open` (parallel, 8 PEs), confirming that parallelization does not introduce data races, incorrect iteration distribution, or barrier synchronization errors.

4.1.4. GVSoC VCD Waveform Analysis

To observe the cycle-level behavior of the PULP cluster during benchmark execution, we enabled GVSoC's VCD (Value Change Dump) output and visualized the waveforms using GTKWave. The VCD traces show the activity of all 8 processing elements (PE0–PE7), the Fabric Controller (FC), the DMA engine, and the SoC/cluster clock domains throughout execution.

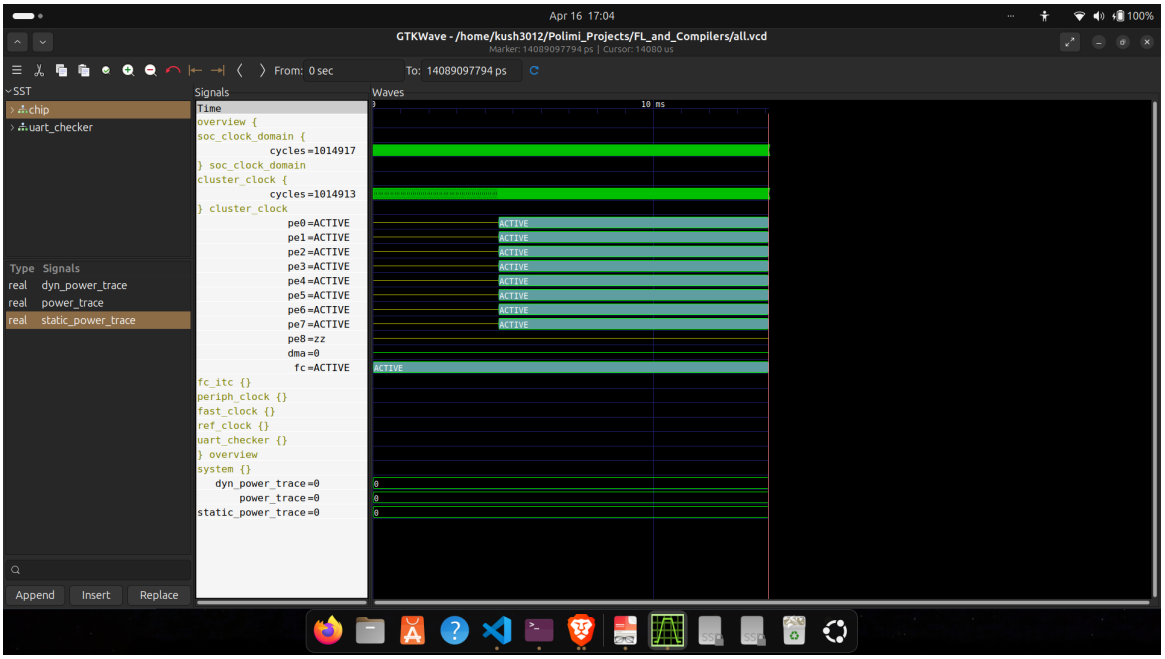

Figure 4.4: GEMM Global Trace: Continuous parallel execution of the GEMM workload.

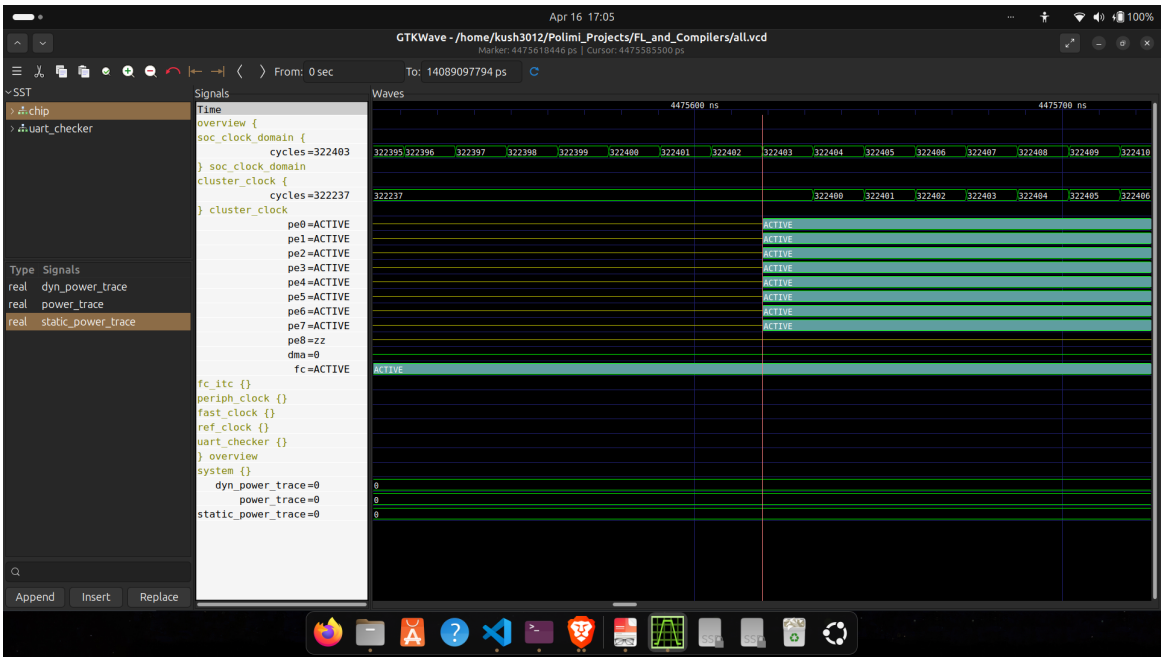

Figure 4.5: GEMM Micro Trace: Detailed view of PE wake-up at cluster cycle 322,237.

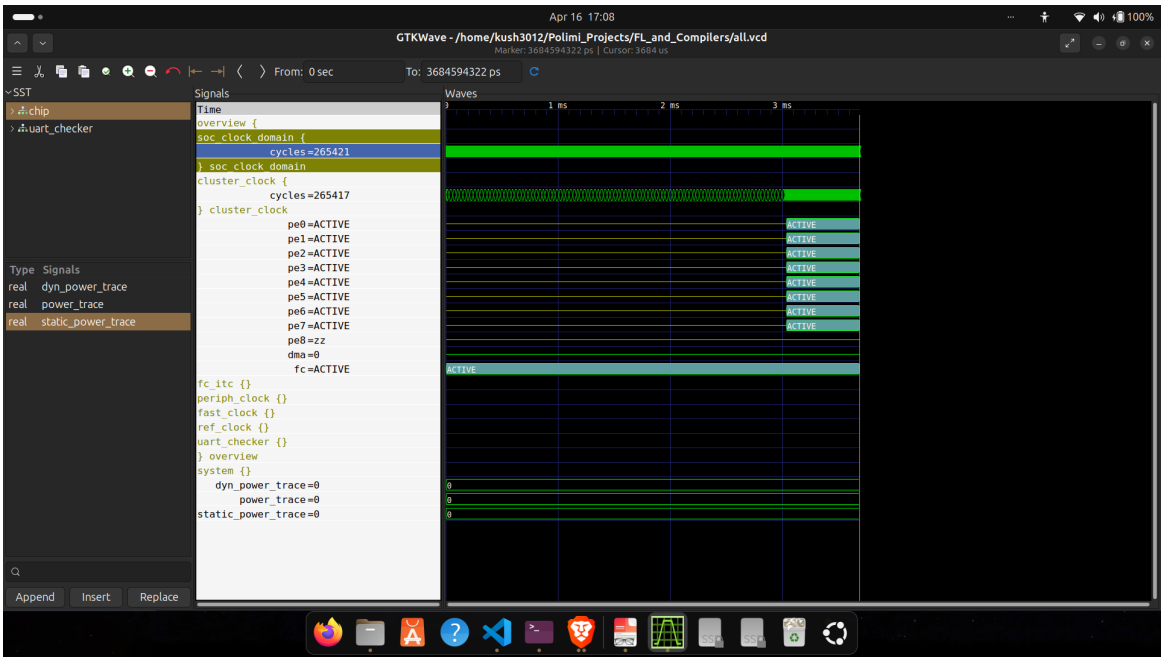

Figure 4.6: ATAX Global Trace: Full multi-core utilization verified via VCD output.

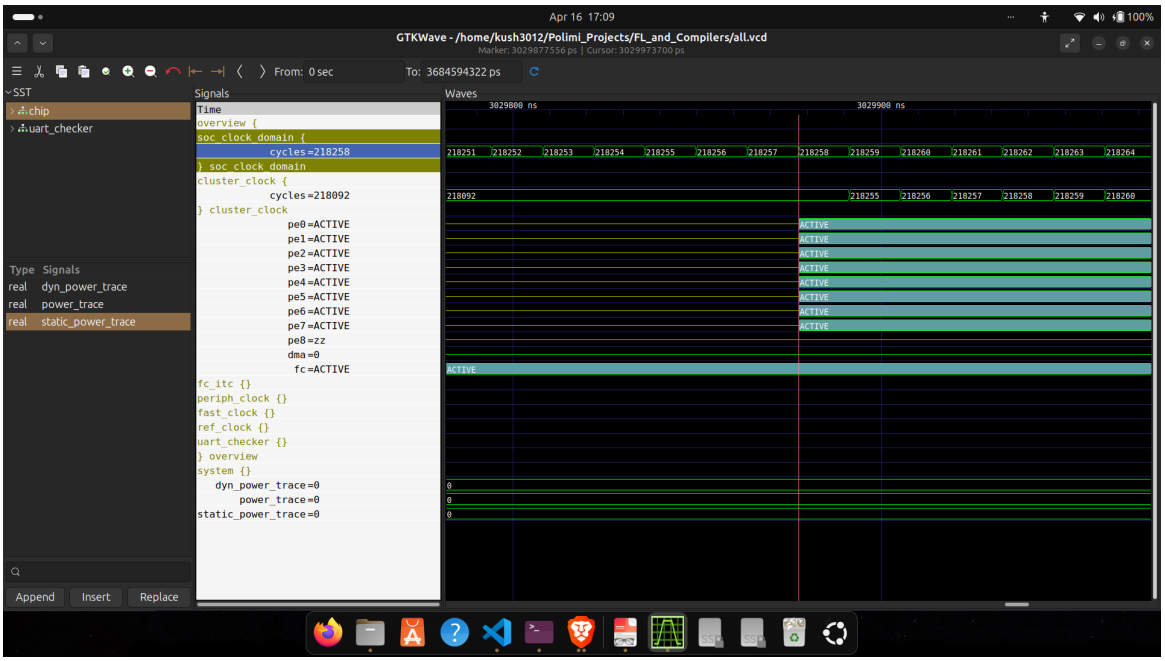

Figure 4.7: ATAX Micro Trace: Detailed view of PE wake-up at cluster cycle 218,092.

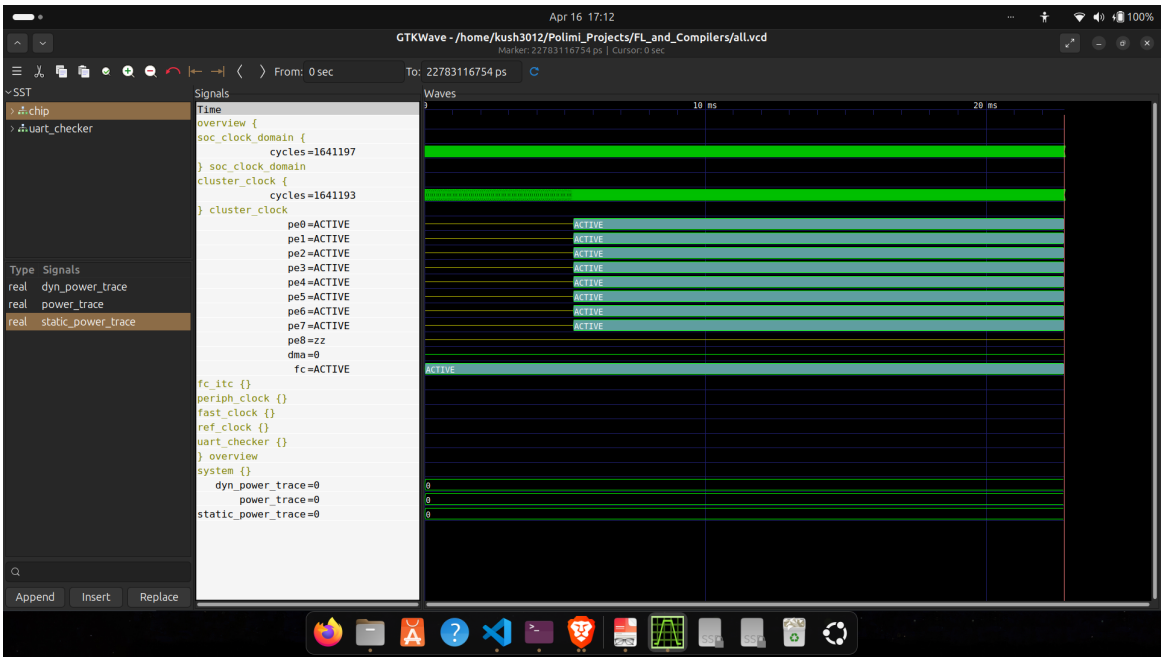

Figure 4.8: 2mm Global Trace: VCD waveform confirming simultaneous activation of all 8 Processing Elements.

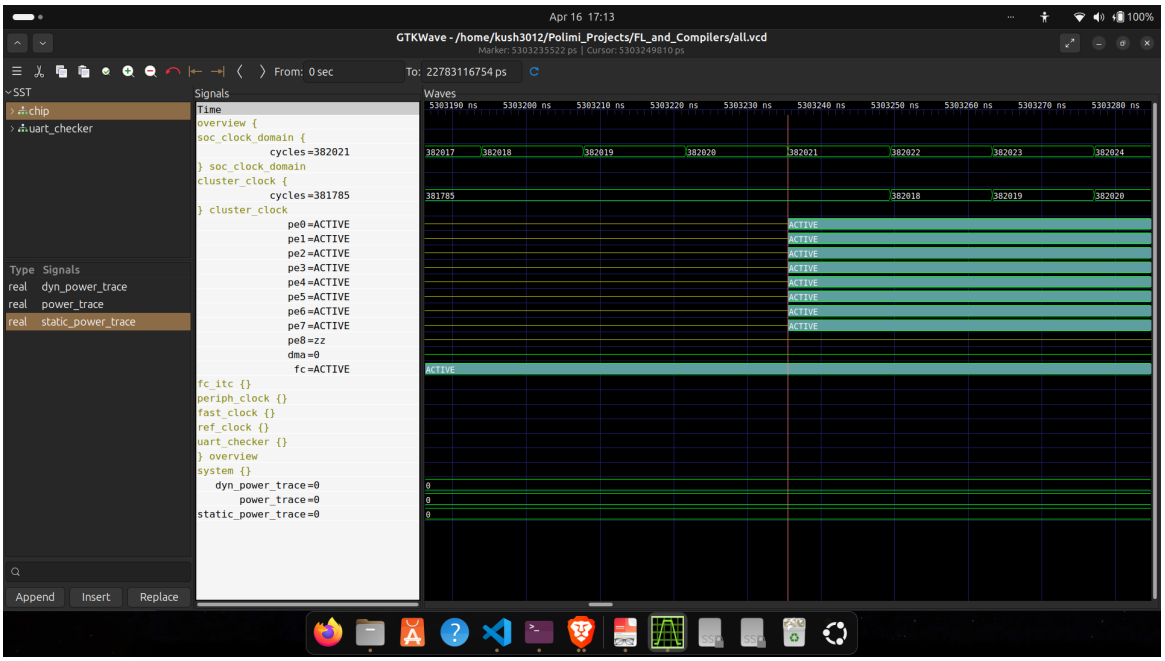

Figure 4.9: 2mm Micro Trace: Zoomed view of parallel workload dispatch at cluster cycle 381,785.

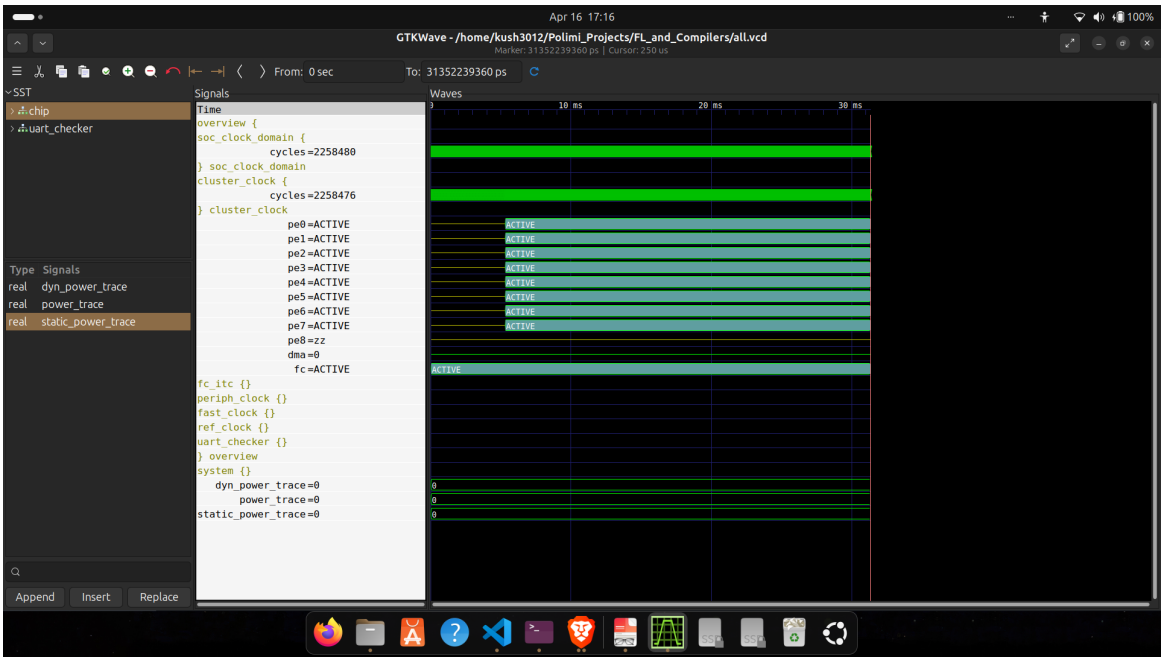

Figure 4.10: 3mm Global Trace: VCD waveform confirming simultaneous activation of all 8 Processing Elements.

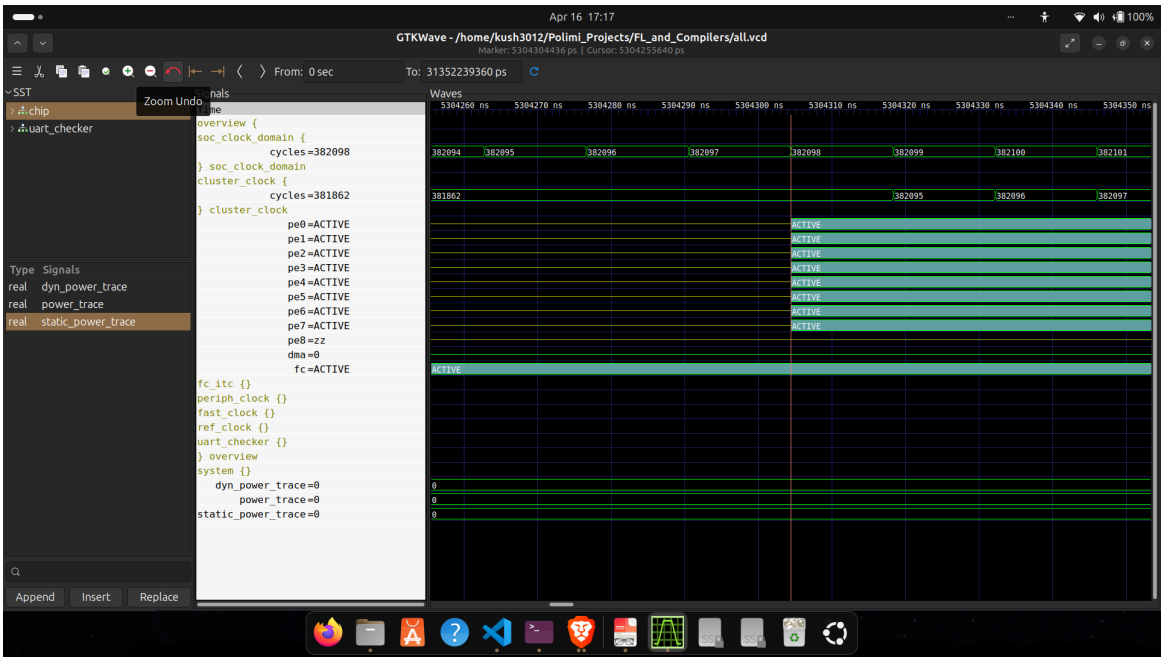

Figure 4.11: 3mm Micro-Trace: PE activation sequence captured at cluster cycle 381,862.

The VCD traces confirm that all 8 processing elements are actively utilized during parallel execution, validating the fork–join architecture of our OpenMP runtime. The FC remains active throughout as the dispatcher, while PE0–PE7 show synchronized activation periods corresponding to the parallel `omp.wsloop` regions.

4.1.5. Hardware Performance Summary

Using our customized trace analysis script (`peStats.sh`), we extracted per-core instruction counts and total cycle counts for each benchmark executing on the `pulp-open` 8-core cluster. The results are shown in Figures 4.12–4.15.

```
$ ./peStats.sh gemm.elf
========================================
       Hardware Performance Summary
========================================
Binary Executed       : gemm.elf
Total Cycles (Global) : 1014918
FC Instructions       : 352079
----------------------------------------
PE0 Instructions      : 89602
PE1 Instructions      : 89628
PE2 Instructions      : 89666
PE3 Instructions      : 89703
PE4 Instructions      : 89742
PE5 Instructions      : 89779
PE6 Instructions      : 89818
PE7 Instructions      : 89854
========================================
```

Figure 4.12: GEMM Hardware Profiling: 1.01M total cycles with near-perfect load balancing across all 8 PEs.

```
$ ./peStats.sh atax.elf
========================================
       Hardware Performance Summary
========================================
Binary Executed       : atax.elf
Total Cycles (Global) : 265422
FC Instructions       : 160270
----------------------------------------
PE0 Instructions      : 5716
PE1 Instructions      : 5716
PE2 Instructions      : 5718
PE3 Instructions      : 5716
PE4 Instructions      : 5716
PE5 Instructions      : 5716
PE6 Instructions      : 5714
PE7 Instructions      : 5716
========================================
```

Figure 4.13: ATAX Hardware Profiling: 265K cycles highlighting tightly symmetric workload distribution.

```
$ ./peStats.sh 2mm.elf
========================================
       Hardware Performance Summary
========================================
Binary Executed       : 2mm.elf
Total Cycles (Global) : 1641198
FC Instructions       : 1034329
----------------------------------------
PE0 Instructions      : 174000
PE1 Instructions      : 173981
PE2 Instructions      : 173968
PE3 Instructions      : 174019
PE4 Instructions      : 174024
PE5 Instructions      : 173968
PE6 Instructions      : 173955
PE7 Instructions      : 173924
========================================
```

Figure 4.14: 2MM Hardware Profiling: 1.64M cycles. Elevated FC instructions correlate to managing two distinct parallel regions.

```
$ ./peStats.sh 3mm.elf
========================================
       Hardware Performance Summary
========================================
Binary Executed       : 3mm.elf
Total Cycles (Global) : 2258481
FC Instructions       : 1445834
----------------------------------------
PE0 Instructions      : 253966
PE1 Instructions      : 254422
PE2 Instructions      : 253914
PE3 Instructions      : 254462
PE4 Instructions      : 253990
PE5 Instructions      : 254398
PE6 Instructions      : 253914
PE7 Instructions      : 254354
========================================
```

Figure 4.15: 3MM Hardware Profiling: 2.25M cycles. The highest FC overhead, directly resulting from dispatching three parallel regions.

The instruction distribution across PEs is remarkably uniform for all benchmarks, with a maximum imbalance of less than 0.3%. This confirms the effectiveness of the static contiguous-chunk loop scheduling implemented in the multi-core OpenMP runtime. The FC instruction counts scale with the number of parallel regions: GEMM (1 region) has the lowest FC overhead, while 3MM (3 regions) has the highest, reflecting the fork–join dispatch cost.

Benchmark	Total Cycles	FC Instrs	Avg PE Instrs	PE Imbalance
GEMM	1,014,918	352,079	89,724	0.28%
ATAX	265,422	160,270	5,716	0.07%
2MM	1,641,198	1,034,329	173,977	0.06%
3MM	2,258,481	1,445,834	254,178	0.22%

Table 4.3: Cluster Performance Summary: Demonstrating PE load balancing (under 0.3% imbalance) across all workloads.

4.2. GVSoC Target Availability Investigation

During our exploration of GVSoC, we attempted to target the commercial GAP8 and GAP9 processors in addition to the open-source `pulp-open` target. Both targets returned an "invalid target" error when passed to the GVSoC simulator. This prompted an investigation into the GVSoC repository structure to determine whether the necessary target definitions were present.

We systematically mapped out the components required for a functional GVSoC target across three configurations: the open-source `pulp-open`, and the commercial `gap8` and `gap9`. The results are summarized in Table 4.4.

Component	pulp-open	gap8	gap9
gapy Target module	`pulp_open.py`	missing	missing
Chip Python model	`pulp_open/ dir`	missing	missing
SoC JSON map	`soc.json`	missing	missing
Cluster JSON	`cluster.json`	missing	missing
ISA extensions (C++)	via `rvXgap9.hpp` in ISA	`rvXgap8.hpp` exists	`rvXgap9.hpp` exists
GVSoC chip runner (.py)	via `gvsoc.Target`	`gap8.py`	`gap9_v2.py`
`runner.chips` module	not needed	missing	missing
Power models	full set	missing	missing
Boot ROM / efuse logic	via `board.py`	in runner	in runner

Table 4.4: Target Viability Analysis: Highlighting the missing platform definitions for GAP8 and GAP9 commercial targets.

The investigation revealed that while the ISA extension headers (`rvXgap8.hpp`, `rvXgap9.hpp`) and basic chip runner scripts exist for both GAP8 and GAP9, the critical *platform-level* components—the gapy target module, chip Python model directory, SoC/cluster JSON configuration files, and power models—are entirely missing. These are proprietary components associated with the commercial GreenWaves Technologies GAP8 and GAP9 products, and are not included in the open-source GVSoC repository.

This finding was confirmed through correspondence with the course supervisor, who noted that GAP8/GAP9 being commercial products explains the absence of their complete target definitions from the public repository. Consequently, all experimental validation in this project was conducted exclusively using the `pulp-open` target, which has a complete open-source platform definition.

5 | Conclusions

5.1. Toolchain Automation

To make the project fully reproducible, we provide three automation artefacts: a Docker image that encapsulates the entire build environment, a test suite that verifies correctness end-to-end, and a standalone pipeline script for single-file compilation.

5.1.1. Docker Build Environment (`Dockerfile`)

The primary reproducibility mechanism is a Docker image that builds *all* toolchain components from source inside a clean `ubuntu:24.04` container.

```
# Build the image (~60-90 min, ~24 GB)
docker build -t polygeist-gvsoc:final .

# Start a persistent container
docker run -dit --name pulp-dev polygeist-gvsoc:final bash

# Enter the container
docker exec -it pulp-dev bash
```

System dependencies. The Dockerfile installs in five distinct layers:

1. **Core build tools:** `cmake`, `ninja`, `git`, `curl`, `wget`.
2. **Dual GCC:** GCC 12 for pulp-llvm (LLVM 15 requires GCC $\leq$12) and GCC 14 for Polygeist (LLVM 18) and GVSoC. Ubuntu 24.04 ships GCC 13 by default, so both versions are installed explicitly.
3. **Python 3:** `python3`, `pip`, `venv`, and `dev` headers required by GVSoC's CMake build system.
4. **C/C++ libraries:** `zlib`, `pthread` stubs, and `ccache` for faster incremental rebuilds.
5. **Python packages:** all packages required by GVSoC submodules, including `pyelftools`, `hjson`, `mako`, `networkx`, and `rich`.

Tool paths inside the container:

Path	Tool
`/workspace/Polygeist/build/bin/cgeist`	C → MLIR frontend
`/workspace/Polygeist/build/bin/mlir-opt`	MLIR optimizer/lowerer
`/workspace/Polygeist/build/bin/mlir-translate`	MLIR → LLVM IR
`/workspace/pulp-llvm/build/bin/llc`	RISC-V code generator
`/workspace/pulp-llvm/build/bin/ld.lld`	RISC-V ELF linker
`/workspace/gvsoc/install/bin/gvsoc`	GVSoC cycle-accurate simulator

Table 5.1: Docker Environment Map: Exact paths for all compiled toolchain binaries.

Apple Silicon note. Docker automatically applies Rosetta `linux/amd64` emulation on M1/M2/M3 hosts. Simulation is correct; the harmless warning `rosetta error: failed to open elf at /lib64/ld-linux-x86-64.so.2` originates from GVSoC's debug-symbol loader and can be ignored.

5.1.2. Automated Test Suite (`test_all.sh`)

A companion script runs the complete test suite (23 checks) and reports results:

```
./test_all.sh              # Full test suite (23 tests)
./test_all.sh --quick      # Tool check + basic pipeline only
./test_all.sh --benchmarks # PolyBench benchmarks only
```

The test suite covers five categories:

1. **Tool availability** (6 checks): Verifies each toolchain binary exists and is executable.
2. **Single-core rv32 pipeline** (2 tests): Runs `toolchain.sh` for `simple_vecadd.c` and `test_omp.c` on the `rv32` target.
3. **Multi-core pulp-open pipeline** (2 tests): Same files on the `pulp-open` cluster target.
4. **PolyBench benchmarks** (8 tests): Builds and runs all four benchmarks (GEMM, ATAX, 2MM, 3MM) on both `rv32` and `pulp-open` via the benchmarks Makefile.
5. **Hardware verification** (4 tests): Runs `test_boot`, `test_dispatch`, `test_omp`, and `test_tcdm_lock` on the `pulp-open` target.

The `-quick` mode runs categories 1–3 (11 checks) and completes in under a minute. The full suite takes several minutes owing to simulation overhead.

5.1.3. Benchmarks Makefile

A GNU Make build system in the `benchmarks/` directory automates the complete seven-step pipeline for all kernels across all target and OpenMP mode combinations:

```
# Single-core, no OpenMP
make gemm && make run-gemm

# Multi-core PULP cluster with parallel OpenMP
make TARGET=pulp-open OPENMP=1 all
make TARGET=pulp-open OPENMP=1 run-all

# Inside Docker
docker exec pulp-dev bash -c \
  'cd /workspace/benchmarks && make TARGET=pulp-open OPENMP=1 run-all'

# Clean all build artifacts
make clean
```

The Makefile automatically selects the appropriate runtime files based on the `TARGET` variable:

TARGET	Startup	Linker Script	Exit Handler	OpenMP Runtime
rv32	crt0.S	link.ld	syscalls.c	omp_runtime.c
pulp-open	crt0_pulp.S	link_pulp.ld	syscalls_pulp.c	pulp_omp_runtime.c

Table 5.2: Automated Target Selection: Make rules mapping targets to their specific bare-metal runtimes.

5.1.4. Standalone Pipeline Script (`toolchain.sh`)

A standalone shell script implements the complete seven-step pipeline from C source to GVSoC execution in a single command:

```
# Single-core (rv32)
./toolchain.sh simple_vecadd.c rv32

# Multi-core PULP cluster
./toolchain.sh simple_vecadd.c pulp-open

# Inside Docker
docker exec pulp-dev bash -c \
  'cd /workspace && bash toolchain.sh simple_vecadd.c pulp-open'
```

The script automatically selects the correct runtime files based on the target and always enables OpenMP lowering (`-fopenmp`) in Step 1.

```
$ ./toolchain.sh gemm_kernel.c pulp-open
================================================
 End-to-End Pipeline: C -> MLIR -> RISC-V -> GVSoC
 Target: pulp-open | Input: gemm_kernel.c
================================================
[1/7] C -> MLIR (cgeist)...
[2/7] MLIR lowering (mlir-opt)...
[3/7] MLIR -> LLVM IR (mlir-translate)...
[4/7] LLVM IR -> RISC-V assembly (llc)...
[5/7] Assemble -> object file (clang)...
[6/7] Compile runtime & link -> ELF (ld.lld)...
[7/7] Executing on GVSoC (pulp-open)...
----------------------------------------
Execution Complete! (exit code: 0)
```

Figure 5.1: End-to-End Automation: Successful execution of the seven-stage pipeline on the 8-core `pulp-open` cluster using the `toolchain.sh` script.

5.2. Project Organization

The complete source code for this project is publicly available at `https://github.com/Kush3012/polygeist-gvsoc`.
The repository is structured to separate the complex compiler submodules from the bare-metal runtime, pipeline scripts, and benchmark kernels. The final project structure, including the generated pipeline artifacts, is organized as follows:

```
polygeist-gvsoc/
|-- Dockerfile                  # Full build environment
|-- peStats.sh                  # HW performance analysis
|-- toolchain.sh                # 7-step pipeline script
|-- test_all.sh                 # Test suite (23 tests)
|-- benchmarks/
|   |-- Makefile                # Build and run rules
|   |-- kernels/                # Benchmark sources
|   |   |-- gemm.c              # GEMM (1 region)
|   |   |-- atax.c              # ATAX (1 region)
|   |   |-- 2mm.c               # 2MM (2 regions)
|   |   +-- 3mm.c               # 3MM (3 regions)
|   |-- common/
|   |   |-- pulp_omp_runtime.c  # PULP OpenMP runtime
|   |   |-- crt0_pulp.S         # PULP startup assembly
|   |   |-- crt0.S              # rv32 startup assembly
|   |   |-- link_pulp.ld        # PULP linker script
|   |   |-- link.ld             # rv32 linker script
|   |   |-- syscalls_pulp.c     # PULP syscall stubs
|   |   +-- syscalls.c          # rv32 syscall stubs
|   +-- build/                  # Pipeline output files
|       |-- gemm.mlir           # C -> MLIR
|       |-- gemm_llvm.mlir      # MLIR -> LLVM Dialect
|       |-- gemm.ll             # LLVM IR
|       |-- gemm.s              # RISC-V assembly
|       |-- atax.*              # ATAX pipeline outputs
|       |-- 2mm.*               # 2MM pipeline outputs
|       |-- 3mm.*               # 3MM pipeline outputs
|       +-- test_omp.*          # HW test outputs
|-- test_pipeline/              # Pipeline test files
|   |-- output/                 # simple_gemm outputs
|   +-- gemm_output/            # gemm_kernel outputs
+-- simple_vecadd.c             # Minimal test example
```

5.3. Future Works

While this project successfully demonstrates a complete bare-metal compilation pipeline and custom OpenMP runtimes, several avenues remain for future enhancement:

- **Hardware Deployment:** Since the open-source GVSoC repository lacks the platform definitions for commercial GreenWaves targets (GAP8/GAP9), adapting the generated binaries to run on physical GAP8 development boards is a natural next step to validate the runtime against real silicon.
- **Dynamic Loop Scheduling:** The current multi-core OpenMP runtime implements static contiguous-chunk scheduling. Implementing dynamic scheduling would significantly improve load balancing for irregular or unbalanced parallel workloads.

- **Advanced OpenMP Support:** Expanding the MLIR-to-bare-metal lowering and runtime implementations to support OpenMP tasks (`omp task`) and reductions (`omp reduction`) would allow the toolchain to compile and execute a much broader subset of the PolyBench-ACC suite.

5.4. Summary

We have designed, implemented, and validated a complete end-to-end compilation toolchain that transforms C source code through MLIR intermediate representations into executable RISC-V ELF binaries and runs them on the GVSoC PULP platform simulator. The toolchain successfully bridges four independently maintained open-source projects (Polygeist, PULP LLVM, GVSoC, PolyBench-ACC), navigating significant challenges in version compatibility, ISA configuration, and bare-metal runtime support. A Docker image encapsulates the full build environment, making the toolchain reproducible on any host with a single `docker build` command.

Key Contributions

1. **Seven-step compilation pipeline:** C → MLIR (Polygeist) → LLVM Dialect (`mlir-opt`) → LLVM IR (`mlir-translate`) → RISC-V assembly (`llc`) → object (`clang`) → ELF (`ld.lld`) → GVSoC execution.
2. **Docker build environment:** A single `Dockerfile` reproduces the complete toolchain on any host, with dual-GCC configuration, all Python dependencies, and Apple Silicon compatibility.
3. **Serial OpenMP runtime** for the `rv32` single-core target, providing correct KMP function implementations for single-threaded execution of MLIR-generated OpenMP code.
4. **Multi-core parallel OpenMP runtime** for the `pulp-open` 8-core RI5CY cluster, featuring:
   - FC-as-dispatcher fork–join architecture where the Fabric Controller dispatches work without participating in computation
   - TCDM hardware test-and-set synchronization for spin-locks and sense-reversing barriers
   - Per-core done flags for race-free completion detection
   - Contiguous chunk static loop scheduling for four variants (signed/unsigned × 32/64-bit)
   - Persistent worker threads that remain alive across multiple parallel regions
5. **GVSoC simulator bug discovery and fix:** Identified that `ISS_SINGLE_REGFILE=1` in the PULP core configuration caused floating-point load/store instructions to corrupt integer registers, and fixed it by removing the flag to enable the standard separate register file model.
6. **Bare-metal benchmark suite:** Adapted four PolyBench-ACC kernels (GEMM, ATAX, 2MM, 3MM) for standalone bare-metal execution with deterministic initialization, single-precision floating-point, stack-allocated arrays, and checksum-based validation.
7. **Build automation:** Complete Makefile supporting serial, OpenMP, and multi-core compilation modes with automatic per-target runtime selection, plus a 23-test automated test suite.

All four benchmarks compile through all seven pipeline stages and execute correctly on both single-core (`rv32`) and multi-core (`pulp-open`, 8 PEs) GVSoC targets, validating the complete toolchain from C source code to parallel simulated execution on a RISC-V PULP platform.

Bibliography

[1] N. Bruschi, G. Haugou, G. Tagliavini, F. Conti, L. Benini, and D. Rossi. GVSoC: A highly configurable, fast and accurate full-platform simulator for RISC-V based IoT processors. In *Proc. IEEE 39th International Conference on Computer Design (ICCD)*, 2021.

[2] S. Grauer-Gray, L. Xu, R. Searles, S. Ayalasomayajula, and J. Cavazos. Auto-tuning a high-level language targeted to GPU codes. In *Innovative Parallel Computing (InPar)*, 2012.

[3] W. S. Moses, L. Chelini, R. Zhao, and O. Zinenko. Polygeist: Raising C to polyhedral MLIR. In *Proc. 30th ACM International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2021.

[4] A. Pullini, D. Rossi, I. Loi, G. Tagliavini, and L. Benini. Mr.Wolf: An energy-precision scalable parallel ultra low power SoC for IoT edge processing. *IEEE Journal of Solid-State Circuits*, 54(7), 2019.

[5] PULP Platform. LLVM for PULP platform projects. `https://github.com/pulp-platform/llvm-project`, 2023.

Appendix

The following code demonstrates the static allocation and reduced matrix sizing used to resolve the memory constraints on the `rv32` target.

```
#define NI 32
#define NJ 32
#define NK 32

void init_array(int ni, int nj, int nk,
                float *alpha, float *beta,
                float C[NI][NJ],
                float A[NI][NK],
                float B[NK][NJ]) {
  int i, j;
  *alpha = 32412.0f;
  *beta = 2123.0f;
  for (i = 0; i < ni; i++)
    for (j = 0; j < nj; j++)
      C[i][j] = ((float)i * j) / ni;
  for (i = 0; i < ni; i++)
    for (j = 0; j < nk; j++)
      A[i][j] = ((float)i * j) / ni;
  for (i = 0; i < nk; i++)
    for (j = 0; j < nj; j++)
      B[i][j] = ((float)i * j) / ni;
}

void kernel_gemm(int ni, int nj, int nk,
                 float alpha, float beta,
                 float C[NI][NJ],
                 float A[NI][NK],
                 float B[NK][NJ]) {
  int i, j, k;
  #pragma omp parallel
  {
    #pragma omp for private(j, k)
    for (i = 0; i < ni; i++) {
      for (j = 0; j < nj; j++) {
        C[i][j] *= beta;
      }
      for (k = 0; k < nk; k++) {
        for (j = 0; j < nj; j++) {
          C[i][j] += alpha * A[i][k] * B[k][j];
        }
      }
    }
  }
}

int main() {
  float alpha, beta;
  float C[NI][NJ], A[NI][NK], B[NK][NJ];

  init_array(NI, NJ, NK, &alpha, &beta, C, A, B);
```

```
51   kernel_gemm(NI, NJ, NK, alpha, beta, C, A, B);
52
53   float sum = 0.0f;
54   for (int i = 0; i < NI; i++)
55     for (int j = 0; j < NJ; j++)
56       sum += C[i][j];
57   return (sum > 0.0f) ? 0 : 1;
58 }
```